

JavaScript - Day 4

Hoisting, Scope & Closures

Student Notes & Interview Preparation Guide

Topics covered: Hoisting | Execution Phases | Temporal Dead Zone | Scope Types | Scope Chain | Lexical Scope | Closures | Real-World Uses of Closures

Topic Overview

In this session, we learned three important JavaScript concepts:

1. Hoisting
2. Scope
3. Closures

These concepts help us understand how JavaScript stores variables, finds variables, and remembers variables.

They are also among the most frequently asked JavaScript interview topics and are heavily used in modern frameworks like React.

1. Hoisting

Why Do We Need Hoisting?

Consider the following code:

```
greet();

function greet() {
  console.log("Hello");
}
```

The function works even though it is called before its declaration. Why? Because of Hoisting.

Understanding hoisting helps us predict the behavior of:

- Variables
- Functions
- Scope
- Execution Order

What is Hoisting?

Definition

Hoisting is a JavaScript mechanism where variable declarations and function declarations are moved to the top of their containing scope before code execution.

Before executing code, JavaScript scans the entire code and allocates memory for variables and functions.

Because of this behavior, variables and functions may appear accessible before their declaration in the code.

JavaScript Execution Phases

JavaScript executes code in two phases.

Phase 1: Memory Creation Phase

JavaScript scans the entire code. It allocates memory for:

- Variables
- Functions

Example:

```
var a = 10;

function greet() {
  console.log("Hello");
}
```

During Memory Creation Phase:

```
a = undefined

greet = complete function
```

Phase 2: Execution Phase

JavaScript executes code line by line.

```
a = 10;
```

Now the actual value is assigned.

Important Rule

Only declarations are hoisted, not initializations.

JavaScript moves:

```
var a;
```

But it does NOT move:

```
a = 10;
```

Variable Hoisting

Example

```
console.log(a);

var a = 5;

console.log(a);
```

Output:

```
undefined
5
```

How JavaScript Interprets It

```
var a;
```

```
console.log(a);  
  
a = 5;  
  
console.log(a);
```

Explanation

During memory creation:

```
a = undefined
```

The declaration is hoisted, but the value assignment happens later during execution.

Therefore:

```
console.log(a);
```

prints:

```
undefined
```

Initialization is NOT Hoisted

Example

```
console.log(b);  
  
var b = 10;  
  
console.log(b);
```

Output:

```
undefined  
10
```

Explanation

Only this part is hoisted:

```
var b;
```

This part is NOT hoisted:

```
b = 10;
```

Therefore the variable contains undefined until the assignment is executed.

Hoisting with var

Example 1

```
a = 10;  
  
console.log(a);  
  
var a;
```

Output:

```
10
```

Why?

JavaScript internally treats it as:

```
var a;  
  
a = 10;  
  
console.log(a);
```

Example 2

```
var b;  
  
console.log(b);  
  
b = 10;
```

Output:

```
undefined
```

Hoisting with let and const

Many beginners think: let and const are not hoisted. This is incorrect. They are hoisted, but they behave differently.

Example

```
console.log(c);  
  
let c = 20;
```

Output:

```
ReferenceError
```

Example

```
console.log(d);  
  
const d = 30;
```

Output:

```
ReferenceError
```

Temporal Dead Zone (TDZ)

What is TDZ?

Variables declared using `let` and `const` are hoisted, but access to them is blocked until the declaration line is reached.

This blocked period is called the Temporal Dead Zone (TDZ).

Simple Definition

The time between entering a scope and initializing a `let/const` variable is called the Temporal Dead Zone.

Visual Representation

```
Scope Starts  
  ↓  
console.log(age) ✗  
  ↓  
let age = 20  
  ↑  
TDZ Ends Here
```

Example

```
c = 10;  
  
let c;
```

Output:

```
ReferenceError
```

Because JavaScript already knows about `c`, but access is blocked until the declaration line.

Function Hoisting

Function declarations are completely hoisted.

Example

```
greet();

function greet() {
  console.log("Hello");
}
```

Output:

```
Hello
```

How JavaScript Interprets It

```
function greet() {
  console.log("Hello");
}

greet();
```

Explanation

During memory creation:

```
greet = complete function
```

Therefore the function can be called before its declaration.

Function Expression Hoisting

Example

```
greet();

const greet = function() {
  console.log("Hello");
};
```

Output:

```
ReferenceError
```

Explanation

The variable `greet` is declared using `const`. Therefore it remains inside the TDZ until initialization.

So the function cannot be called before declaration.

Best Practice

Although hoisting allows access before declaration:

- ✓ Declare variables before using them.
- ✓ Keep function declarations above usage whenever possible.

This improves readability and avoids confusion.

Interview Questions

4. What is Hoisting in JavaScript?
 5. Why does Hoisting occur?
 6. What is the difference between declaration and initialization?
 7. Explain JavaScript Execution Phases.
 8. Is let hoisted?
 9. Is const hoisted?
 10. What is TDZ?
 11. Why does var return undefined but let throws an error?
 12. Why are function declarations hoisted?
-

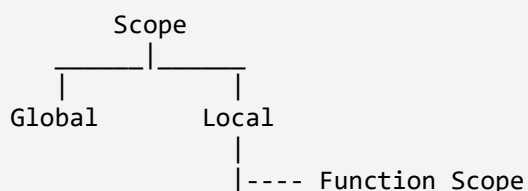
2. Scope

What is Scope?

Scope determines which variables can be accessed from a particular part of the code.

In simple words: scope defines the visibility and accessibility of variables.

Types of Scope



```
|  
|---- Block Scope
```

Global Scope

Variables declared outside all functions and blocks belong to global scope.

Example

```
let collegeName = "Veltech College";  
  
function showCollege() {  
    console.log(collegeName);  
}
```

Output:

```
Veltech College
```

Explanation

The function can access variables from the global scope.

Function Scope

Variables declared inside a function can only be accessed inside that function.

Example

```
function classroom() {  
    let student = "Rahul";  
    console.log(student);  
}
```

Output:

```
Rahul
```

Accessing Outside Function

```
console.log(student);
```

Output:

```
ReferenceError
```

Block Scope

Variables declared using `let` and `const` are block scoped.

Example

```
if (true) {  
  let age = 20;  
}  
  
console.log(age);
```

Output:

```
ReferenceError
```

var and Block Scope

Example

```
if (true) {  
  var age = 20;  
}  
  
console.log(age);
```

Output:

```
20
```

Why?

`var` is function-scoped. It ignores block scope. If no function exists, it behaves like a global variable.

Scope Chain

When JavaScript tries to find a variable, it searches in this order:

13. Current Scope
14. Parent Scope
15. Global Scope

This searching process is called the Scope Chain.

Lexical Scope

What is Lexical Scope?

Lexical Scope means a function can access variables from the scope where it was defined, not where it was executed.

Important Rule

Scope depends on where a function is defined, not where it is executed.

Example 1

```
let city = "Vizag";

function parent() {

  let name = "Yathe";

  function child() {

    console.log(city);
    console.log(name);

  }

  child();
}
```

Output:

```
Vizag
Yathe
```

Why?

The child function can access:

- Its own scope
- Parent scope
- Global scope

Example 2

```
let num = 10;

function first() {
```

```
let num = 20;

function second() {
  console.log(num);
}

second();
}
```

Output:

20

Nearest Scope Wins

JavaScript always checks the nearest variable first.

```
let num = 10;

function first() {

  let num = 20;

  function second() {
    console.log(num);
  }
}
```

Output:

20

because the nearest variable is found first.

Interview Questions

16. What is Scope?
17. What is Global Scope?
18. What is Local Scope?
19. What is Function Scope?
20. What is Block Scope?
21. Why is var not block scoped?
22. What is Scope Chain?
23. What is Lexical Scope?
24. What does "Nearest Scope Wins" mean?

3. Closures

Why Do We Need Closures?

Suppose we want:

- Private Variables
- Data Encapsulation
- Data Privacy
- State Management

JavaScript does not provide true private variables in normal functions. Closures help solve this problem.

What is a Closure?

Definition

A Closure is a function that remembers the environment or scope in which it was created.

Even after the outer function has finished execution, the inner function still retains access to variables from its lexical scope.

Interview Definition

A Closure is created when a function remembers variables from its outer scope even after the outer function has returned.

Closure Formula

```
Closure
=
Function
+
Lexical Environment
```

Understanding Closures Through Lexical Scope

Closures are built on top of Lexical Scope.

Because a function already has access to its outer scope, JavaScript can preserve that scope even after the outer function finishes execution.

This preserved scope creates a Closure.

Basic Closure Example

```
function outer() {  
    let name = "Yathe";  
  
    function inner() {  
        console.log(name);  
    }  
  
    return inner;  
}  
  
const result = outer();  
  
result();
```

Output:

```
Yathe
```

What Happens Internally?

Step 1

```
outer();
```

creates:

```
name = "Yathe"
```

Step 2

The inner function is returned.

```
return inner;
```

Step 3

Normally the outer function finishes execution.

Step 4

JavaScript keeps the variable alive because:

```
inner()
```

still needs access to it. This creates a Closure.

Another Closure Example

```
function outerFunction(outerVariable) {  
    return function innerFunction(innerVariable) {  
        console.log("Outer Variable:", outerVariable);  
        console.log("Inner Variable:", innerVariable);  
    };  
}
```

Usage:

```
const newFunction = outerFunction("outside");  
  
newFunction("inside");
```

Output:

```
Outer Variable: outside  
Inner Variable: inside
```

Counter Example (Most Important)

```
function createCounter() {  
    let count = 0;  
  
    return function() {  
        count++;  
        return count;  
    };  
}
```

Usage:

```
const counter = createCounter();  
  
console.log(counter());  
console.log(counter());  
console.log(counter());
```

Output:


```
1  
2  
3
```

Why?

The inner function remembers count even after the outer function finishes execution.

Login Tracker Example

```
function loginTracker() {  
    let attempt = 0;  
    return function() {  
        attempt++;  
        console.log(`Attempt ${attempt}`);  
    };  
}
```

Usage:

```
const login = loginTracker();  
  
login();  
login();  
login();
```

Output:

```
Attempt 1  
Attempt 2  
Attempt 3
```

How Closures Are Used

1. Private Variables

Closures can hide data from direct access. Examples:

- Bank Balance
- User Information
- Account Details

2. Data Encapsulation

Closures help bundle data and functions together while restricting direct access. This improves security and maintainability.

3. State Management

Closures are heavily used for maintaining state. Examples:

- Counters
- Like Counts
- Cart Counts
- Page Visits

4. Event Handling

Closures are commonly used inside `addEventListener()` callbacks.

```
addEventListener()
```

5. Callbacks and Asynchronous Operations

Closures help preserve data when these execute later:

```
setTimeout()  
setInterval()  
Promises  
API Calls
```

6. JavaScript Frameworks

Many modern libraries and frameworks internally use closures for state management. Examples:

- React
- Redux
- Zustand
- Vue
- Angular

Common Beginner Mistakes

Assuming var is block scoped

✗ Incorrect assumption

```
if(true){  
  var x = 10;  
}
```

```
}  
  
console.log(x);
```

Output:

```
10
```

Accessing let/const before declaration

```
console.log(age);  
  
let age = 20;
```

Produces a TDZ error.

Thinking Closures Copy Variables

Closures do not copy variables. They keep a reference to the original variable.

Thinking Outer Variables Are Destroyed Immediately

If an inner function still needs them, JavaScript keeps them alive.

Thinking Every Nested Function is a Closure

A nested function becomes useful as a closure when it continues to access outer variables after the outer function has finished execution.

Interview Questions

Hoisting

25. What is Hoisting?
26. What are JavaScript Execution Phases?
27. What is TDZ?
28. Difference between declaration and initialization?
29. Why does var return undefined?
30. Are let and const hoisted?
31. Why are function declarations hoisted?

Scope

32. What is Scope?
33. Types of Scope?
34. What is Global Scope?
35. What is Function Scope?

- 36. What is Block Scope?
- 37. What is Scope Chain?
- 38. What is Lexical Scope?
- 39. What does "Nearest Scope Wins" mean?

Closures

- 40. What is a Closure?
 - 41. Why do we need Closures?
 - 42. Explain Closure = Function + Lexical Environment.
 - 43. How do Closures provide Data Privacy?
 - 44. What is Data Encapsulation?
 - 45. How are Closures used in Counters?
 - 46. How are Closures used in Event Handling?
 - 47. How are Closures used in State Management?
 - 48. Give real-world uses of Closures.
 - 49. Why are Closures important in React?
-

Key Takeaways

- ✓ Hoisting moves declarations to the top before execution.
- ✓ JavaScript executes code in two phases: Memory Creation Phase and Execution Phase.
- ✓ Only declarations are hoisted, not initializations.
- ✓ `var` is hoisted and initialized with `undefined`.
- ✓ `let` and `const` are hoisted but remain inside the Temporal Dead Zone (TDZ).
- ✓ Scope determines where variables can be accessed.
- ✓ Types of Scope: Global Scope, Function Scope, Block Scope.
- ✓ JavaScript uses the Scope Chain to find variables.
- ✓ Lexical Scope means a function accesses variables based on where it is defined.
- ✓ Nearest Scope always wins.
- ✓ A Closure is a Function + Lexical Environment.
- ✓ Closures remember variables from outer scopes even after the outer function returns.
- ✓ Closures are widely used for Private Variables, Data Encapsulation, Data Privacy, State Management, Event Handling, Callbacks, and Asynchronous Operations.
- ✓ Modern frameworks like React heavily rely on closure concepts internally.